



Web Fundamentals

WEF1VO

Teil 10: JavaScript DOM

Medientechnik und -design | WS 2025/26

DOM

Das Document Object Model

DOM

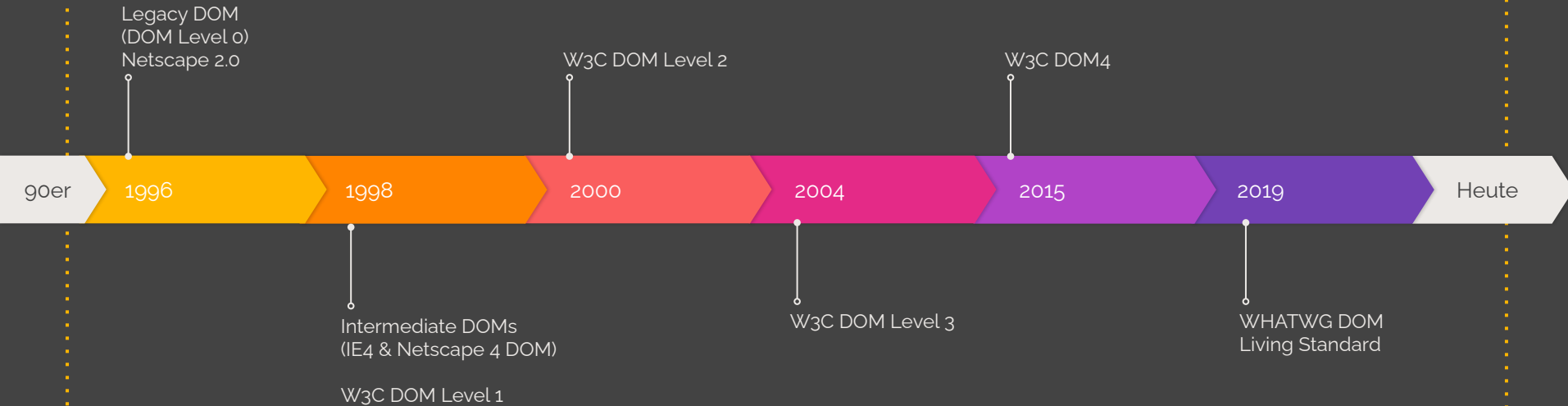
Eine **Schnittstelle** (API).

Bildet die HTML-Struktur in einer Baumstruktur ab.

Zugriff über **Methoden und Eigenschaften** um Inhalte, Struktur und Design dynamisch zu verändern.



DOM Timeline





Legende:

Document

DocumentType

Element

Attr

Text

DOM-Bau

m

Knoten
(Typ Node)

document

<!DOCTYPE
html>

<html>

lang="de"

<head>

<body>

<meta>

charset="utf-8"

<title>

"DOM-
Beispiel"

<h1>

id="title"

"Ein HTML-
Dokument"

<p>

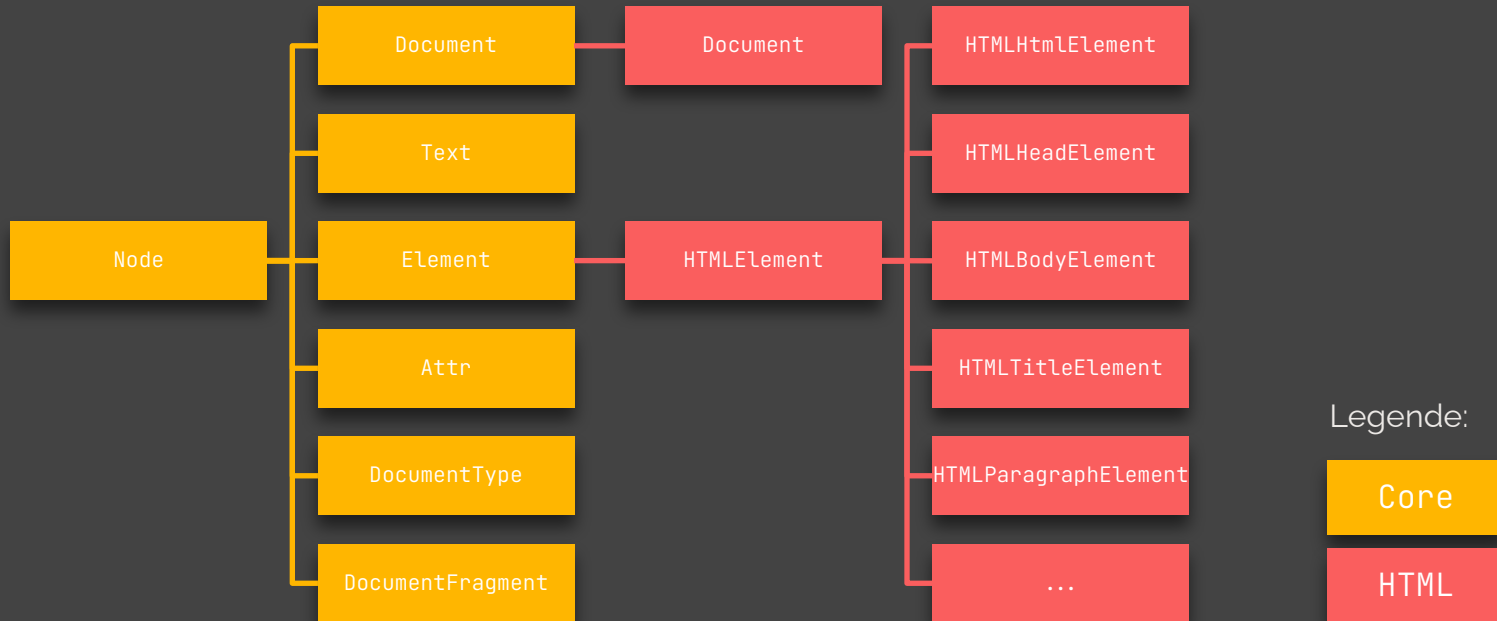
"Das ist
ein "

"einfaches"

" Dokument"

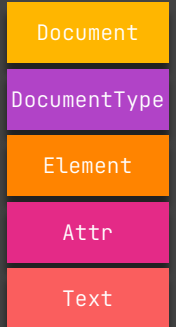
```
>>> <!DOCTYPE html>
>>> <html lang="de">
>>> <head>
>>> <meta charset="utf-8">
>>> <title>DOM-Beispiel</title>
>>> </head>
>>> <body>
>>> <h1 id="title">Ein HTML-Dokument</h1>
>>> <p>Das ist ein <em>einfaches</em> Dokument</p>
>>> </body>
>>> </html>
```

DOM Core & HTML



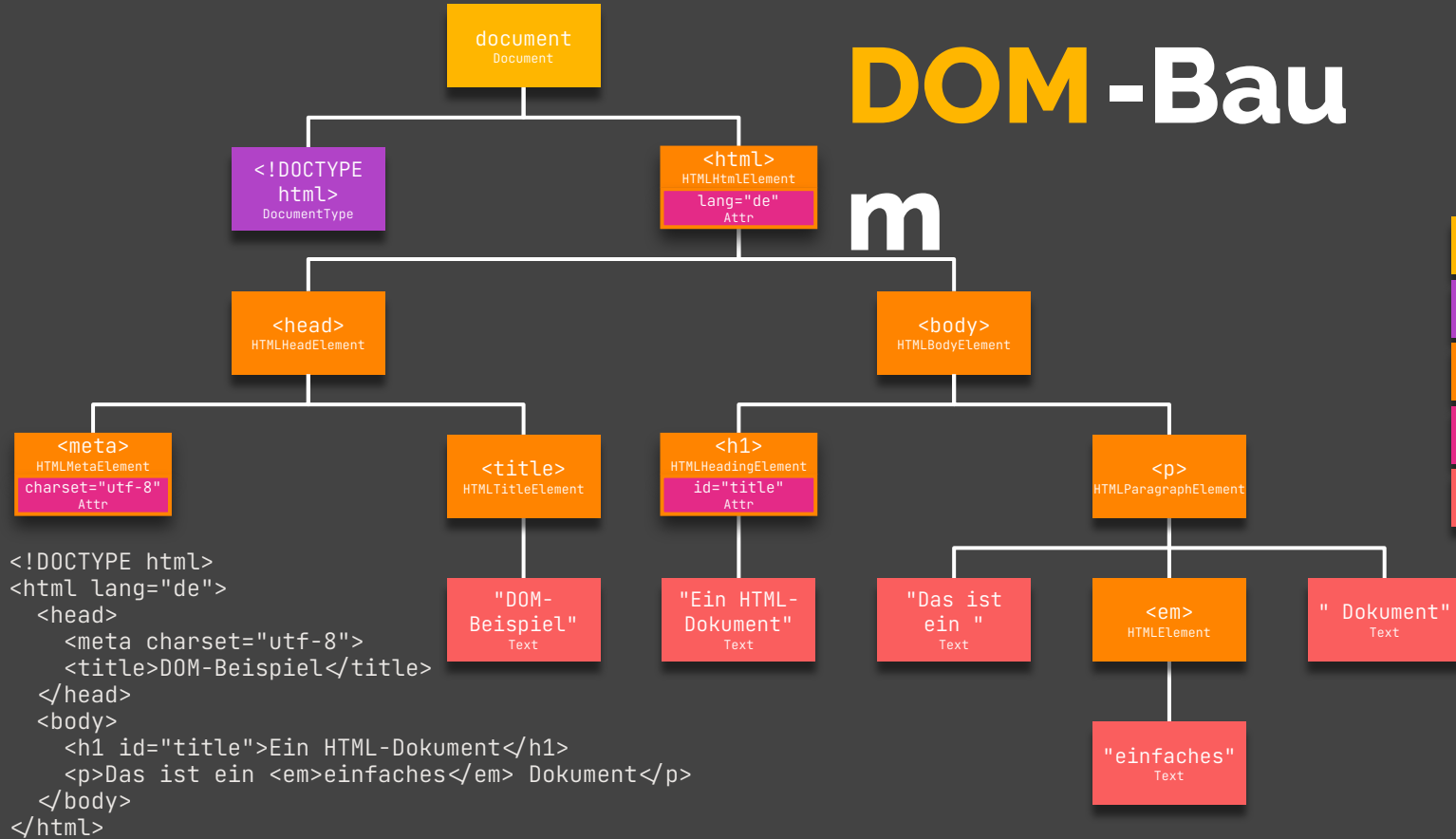


Legende:



DOM-Bau

m



Dokumentmanipulation

Mit Hilfe des DOM



Typ Node

Eigenschaften

nodeName
nodeValue
textContent
nodeType
firstChild, lastChild
nextSibling, previousSibling
parentNode, parentElement
childNodes

Methoden

insertBefore(newChild, refChild)
replaceChild(newChild, oldChild)
appendChild(newChild)
removeChild(oldChild)
hasChildNodes()
contains(node)
cloneNode(deep)

Node Beispiele



```
<div><h1 id="title">Überschrift</h1><p>Hallo</p></div>
```

```
const theDiv = document.querySelector("div");
console.log("Name des Knotens:", theDiv.nodeName); // Ausgabe: "DIV"
console.log("Typ des Knotens:", theDiv.nodeType); // Ausgabe: 1
const h1 = theDiv.firstChild;
console.log("Erstes Kind (h1):", h1);
const p = theDiv.lastChild;
console.log("Letztes Kind (p):", p);
theDiv.replaceChild(p, h1);
console.log("Nach replaceChild (h1 ist weg):", theDiv.innerHTML);
theDiv.removeChild(p);
console.log("Nach removeChild (leer):", theDiv.innerHTML);
console.log("Hat noch Kinder?", theDiv.hasChildNodes()); // Ausgabe: false
```



```
<div>␣
  ␣␣<h1
    ␣␣id="title">Überschrift</h1>␣
    ␣␣<p>Hallo</p>␣
  </div>
```

Typ Document



Eigenschaften

URL / documentURI

characterSet

contentType

compatMode

Methoden

getElementById(id)

getElementsByTagName(tagName)

getElementsByClassName(tagName)

querySelector(selector)

querySelectorAll(selector)

createElement(tagName)

createTextNode(data)

createAttribute(name)

Document Beispiele



```
<div id="content">
```

```
  <p class="red underline">Red and Underlined</p>
```

```
</div>
```

Liefern HTMLCollection
mit Elementen!

```
const content = document.getElementById("content");
```

```
const bodyList = document.getElementsByTagName("body");
```

```
const body = bodyList[0];
```

```
const redUnder = document.getElementsByClassName("red underline");
```

Alternative (NodeList):
document.querySelectorAll(".red
.underline");

```
const pInDiv = document.querySelector("div p");
```

```
const newDiv = document.createElement("div");
```

```
const text = document.createTextNode("Hallo Welt!");
```

```
const idAttr = document.createAttribute("id");
```



Typ Document (HTML)

Eigenschaften

title

head

body

forms

images

cookie

location

Methoden

getElementsByName(name)

~~open()~~

~~close()~~

~~write(text)~~

~~writeln(text)~~

Document (HTML)



Beispiele

```
<form action="mailto:mail@example.com">  
  <input type="radio" name="favorite" value="WEF1" id="wef"><br>  
  <input type="radio" name="favorite" value="MCO1" id="mco"><br>  
  <input type="radio" name="favorite" value="GEC1" id="gec"><br>  
  <button type="submit">Senden</button>  
</form>
```

```
console.log("Seitentitel:", document.title);  
console.log("Body-Referenz:", document.body);  
const favorites = document.getElementsByName("favorite");  
console.log("Gefundene Radio-Buttons:", favorites.length);  
console.log("Anzahl Formulare:", document.forms.length);  
console.log("Anzahl Bilder:", document.images.length);  
document.cookie = "lecture=WEF1; SameSite=Lax";  
console.log("Alle Cookies:", document.cookie);  
console.log("Aktuelle URL:", document.location.href);
```

HTMLCollection mit
Elementen



Typ Element

Eigenschaften

tagName

className

classList

id

innerHTML

firstElementChild,

lastElementChild

nextElementSibling,

previousElementSibling

Methoden

getAttribute(name)

setAttribute(name, value)

removeAttribute(name)

hasAttribute(name)

setAttributeNode(name)

getAttributeNode(name)

remove()

querySelector(),
getElementsByClassName() etc.

Element Beispiele



```
const heading = document.createElement("h1");
heading.textContent = "Überschrift";

heading.setAttribute("id", "important");
heading.id = "normal";
console.log("ID geändert:", heading.getAttribute("id"));
heading.className = "blue"; // Setzt die Klasse blue
heading.classList.add("underline"); // Fügt underline hinzu
if (heading.hasAttribute("id")) {
    heading.removeAttribute("id");
}

document.body.appendChild(heading);
```




Typ

HTML Element

Eigenschaften

title

lang

dir

style

accessKey

tabIndex

HTMLElement Beispiele



```
const divElem = document.createElement("div");  
divElem.textContent = "Hallo Welt!";
```

```
divElem.lang = "de"; // Sprache auf Deutsch stellen  
divElem.style.marginTop = "2em"; // Oberen Rand auf 2em stellen
```

```
console.log("Konfiguriertes Element:", divElem); // Element in Konsole ansehen
```

```
document.body.appendChild(divElem);
```



Subtypen von HTMLElement

Eigenschaften

HTMLLinkElement

HTMLFormElement

HTMLInputElement

HTMLImageElement

etc.

Methoden

href, target, rel etc.

action, method, name etc.

name, value, size etc.

src, alt, width, height etc.

etc.

HTMLInputElement Bsp.



```
const input = document.createElement("input");
```

```
input.type = "password";
```

```
input.autocomplete = "new-password";
```

```
input.placeholder = "Geheimcode eingeben";
```

```
input.value = "123456";
```

```
input.disabled = true;
```

```
console.log("Aktueller Wert:", input.value);
```

```
console.log("Ist deaktiviert:", input.disabled);
```

```
form.appendChild(input);
```

```
document.body.appendChild(form);
```

Typ Attr

Eigenschaften

name

value



Attr Beispiele



```
const divElem = document.createElement("div");  
divElem.textContent = "An ordinary div";
```

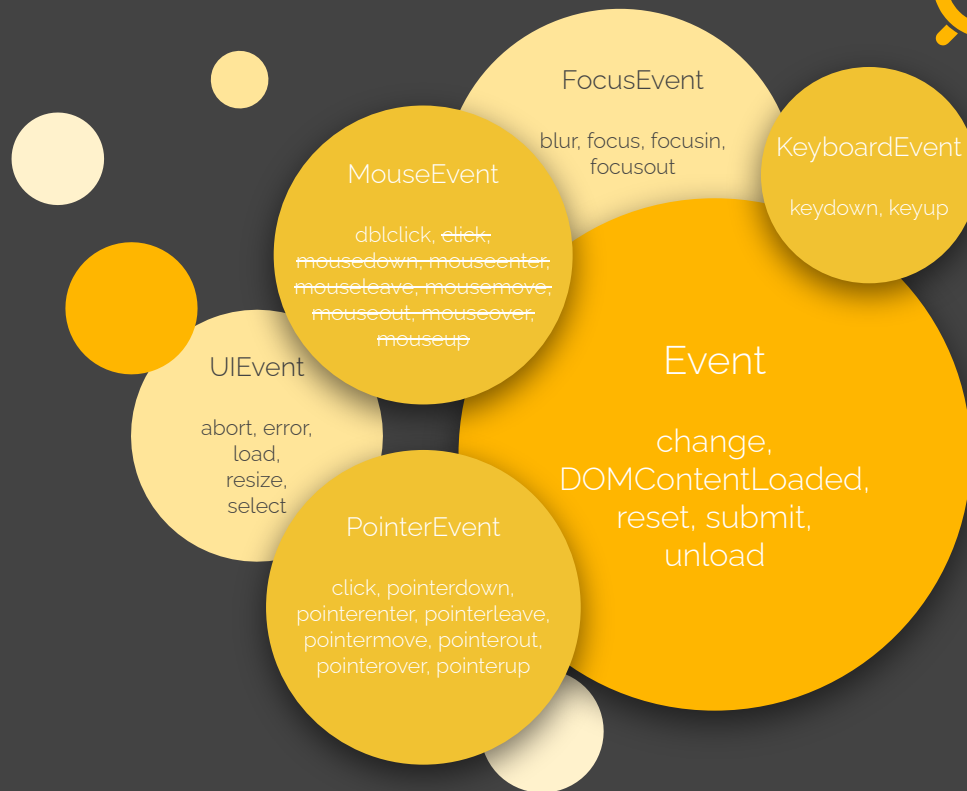
```
const titleAttr = document.createAttribute("title");  
titleAttr.value = "Some Container"; // Inhalt setzen  
console.log("Attribut-Wert:", titleAttr.value);
```

```
divElem.setAttributeNode(titleAttr); // Attribut anhängen  
document.body.appendChild(divElem); // Element ins Dokument hängen  
console.log("HTML mit Attribut:", divElem.outerHTML);
```

Events

Auf Ereignisse im Browser reagieren

Verschiedene Typen von Events im Browser





Verarbeiten von Events



Element auswählen

Mittels DOM-Methoden wird das Element gesucht und ausgewählt, bei dem eine Interaktion stattfinden soll (z.B. Button).



Event spezifizieren

Es wird angegeben, auf welche Art von Event reagiert werden soll (z.B. Mausklick).



Verarbeitung angeben

Es wird eine Funktion definiert, die aufgerufen wird, wenn der Event auftritt (z.B. Alert-Box anzeigen).



Events **im Code** abfangen

HTML-Event-Handler

HTML-Attribute in der Form "on" + Event-Name, z.B. **onclick**, die eine Funktion aufrufen.

DOM-Event-Handler

JavaScript-Attribute in der Form "on" + Event-Name, z.B. **onclick**, die eine Funktion aufrufen.

DOM-Event-Listener

Funktionen zum Verarbeiten von Events werden mit der Methode **addEventListener()** zugewiesen.

HTML-Event-Handler



JavaScript:

```
function alertMe() {  
    window.alert("Event-Handler aufgerufen!");  
}
```

HTML:

```
<form action="mailto:me@email.com" method="post">  
    <h1>Als HTML-Event-Handler</h1>  
    <p>Event-Handler durch HTML-Attribut "onclick" hinzugefügt</p>  
    <button id="button1" type="button" onclick="alertMe()">Button 1</button>  
</form>
```

DOM-Event-Handler



JavaScript:

```
function alertMe() {  
    alert("Event-Handler aufgerufen!");  
}
```

```
const btn2 = document.getElementById("button2");  
btn2.onclick = alertMe;
```

```
btn2.onclick = function () {  
    alertMe();  
    alertMe();  
};
```

HTML:

```
<form action="mailto:me@email.com" method="post">  
    <h1>Als DOM-Event-Handler</h1>  
    <p>Event-Handler mittels JavaScript-Eigenschaft hinzugefügt</p>  
    <button id="button2" type="button">Button 2</button>  
</form>
```



DOM-Event-Listener



JavaScript:

```
function alertMe() {  
    alert("Event-Listener aufgerufen!");  
}  
  
const btn3 = document.getElementById("button3");  
btn3.addEventListener("click", alertMe);
```

HTML:

```
<form action="mailto:me@email.com" method="post">  
    <h1>Als DOM-Event-Listener</h1>  
    <p>Event-Listener durch addEventListener() hinzugefügt</p>  
    <button id="button3" type="button">Button 3</button>  
</form>
```

```
btn3.removeEventListener(  
    "click", alertMe);
```



Standardaktion stoppen



JavaScript:

```
function showURL(event) {  
  const link = document.querySelector("a");  
  window.alert(link.href);  
  event.preventDefault();  
}
```

HTML:

```
<a href="https://www.fh-ooe.at/">Ein Link</a>  
<script>  
  const link = document.querySelector("a");  
  link.addEventListener("click", showURL);  
</script>
```

2x derselbe
Aufruf.
Geht das besser?

Listener-Element -Zugriff



JavaScript:

```
function showURL(event) {  
    window.alert(event.currentTarget.href);  
    event.preventDefault();  
}
```

HTML:

```
<a href="https://www.fh-ooe.at/">Ein Link</a>  
<a href="https://orf.at/">Noch ein Link</a>  
<script>  
    const links = document.querySelectorAll("a");  
    for (const link of links) {  
        link.addEventListener("click", showURL);  
    }  
</script>
```

Auf's Laden warten



```
<!DOCTYPE html>
<html lang="de">
  <head>
    <meta charset="utf-8">
    <title>Unterschied: DOMContentLoaded vs. load</title>
    <script src="domcontentloaded-load.js"></script>
  </head>
  <body>
    <div id="status">Warte auf Events...</div>
    
    <button id="actionButton">Klick mich</button>
  </body>
</html>
```


Auf's Laden warten



```
function initInteractions() {
  const statusBox = document.querySelector("#status");
  const btn = document.querySelector("#actionButton");
  const img = document.querySelector("#demoImage");

  statusBox.textContent = "DOMContentLoaded gefeuert: Struktur ist bereit.";
  statusBox.style.backgroundColor = "#e0f7fa"; // Hellblau

  // Listener anhängen möglich, da das HTML-Element existiert
  btn.addEventListener("click", function () {
    window.alert("Button funktioniert, obwohl das Bild evtl. noch lädt!");
  });

  img.addEventListener("click", event => {
    console.log("Bild wurde angeklickt:", event.currentTarget.src);
  });
}

document.addEventListener("DOMContentLoaded", initInteractions);
```

Auf's Laden warten



```
function finalizePage() {  
    const statusBox = document.querySelector("#status");  
    const img = document.querySelector("#demoImage");  
  
    // Erst jetzt kennen wir die echte Größe des Bildes  
    const width = img.naturalWidth;  
    const height = img.naturalHeight;  
  
    statusBox.textContent += ` | Load gefeuert: Bild geladen  
                             (${width}x${height}px).`;   
    statusBox.style.backgroundColor = "#c8e6c9"; // Hellgrün  
    statusBox.style.border = "2px solid green";  
}  
  
window.addEventListener("load", finalizePage);
```

Formulare validieren



HTML:

```
<form action="mailto:john.doe@johndoe.at" method="post" enctype="text/plain"
      id="dataform">
  <input type="text" id="name">
</form>
```

JavaScript:

```
let hasErrors = false;
```

```
function isValid(event) {
  // Validierung der Formularfelder und setzen von hasErrors auf true, wenn ein Fehler passiert ist

  document.querySelector("input").value === "" ? hasErrors = true : hasErrors = false;
  if (hasErrors) {
    event.preventDefault();
  }
}

document.getElementById("dataform").addEventListener("submit", isValid);
```

Auf Änderungen prüfen



HTML:

```
<select id="choice">
  <option value="A">Wert A</option>
  <option value="B">Wert B</option>
  <option value="C">Wert C</option>
</select>
```

JavaScript:

```
function valChange(event) {
  window.alert(`${event.currentTarget.value} wurde ausgewählt`);
}

document.getElementById("choice").addEventListener("change", valChange);
```



Nächstes Mal in WEF1VO

JavaScript Workflow & Tools

Credits

- No credits to show today.

